SmartGlove Comprehensive User Guide

A Complete Manual: Datasets, AI Training, Backend, Hardware & Firebase Integration

1. System Overview

The SmartGlove system is a comprehensive health and safety monitoring platform utilizing wearable hardware (a glove and a chest belt), a central backend processor, and real-time cloud integration. It employs advanced machine learning and specialized heuristic engines to detect clinical anomalies directly from sensor data.

The system is divided into the following key components:

- **Wearable Hardware:** Smart Glove for vitals and tremor tracking; Smart Belt for ECG and Fall Detection.
- **Clinical Datasets:** Curated datasets used to train independent predictive AI models.
- **AI Models:** Five independent models detecting Sepsis, Hypoxemia, Arrhythmia, Parkinson's Tremors, and Stress.
- **Advanced Heuristics:** Real-time deterministic engines including Early Sepsis SIRS Criteria, Silent Hypoxia, Sensor Fusion, SOS Gestures, and Correlation Logic.
- **Backend Engine:** A Flask-based server managing inference, sensor fusion, WebSocket data streams, and database synchronizations.
- **Firebase Cloud:** Live synchronization for remote monitoring and mobile app dashboards.

2. Datasets & Data Sourcing

The AI models are trained on rich clinical datasets sourced directly from Kaggle, providing accurate ground truth for anomaly detection. These datasets are placed inside their respective model folders as CSV files.

Dataset Sources

- **Hypoxemia Dataset:** https://www.kaggle.com/datasets/bioficanalytics/spo2-hypoxemia-dataset
- **Arrhythmia Dataset:** https://www.kaggle.com/datasets/bioficanalytics/ecg-arrhythmia-features-dataset
- **Sepsis Dataset:** https://www.kaggle.com/datasets/bioficanalytics/sepsis-vital-signs-dataset
- **Parkinson's Dataset:** https://www.kaggle.com/datasets/bioficanalytics/parkinsons-tremor-acceleration-dataset

All data undergoes rigorous preprocessing and median imputation before model training to ensure high precision in realtime edge-case environments.

3. Wearable Hardware Systems

The hardware captures high-frequency biometric data at the edge and transmits it securely via Wi-Fi HTTP POST requests.

3.1 The Smart Glove (glove/main.py)

The glove sits on the user's hand and collects peripheral vitals:

- **MAX30102:** Captures Heart Rate and SpO2 via optical pulse oximetry.

• **MLX90614:** Non-contact IR sensor reading surface body temperature.

• **MPU6050 (Glove):** Captures 3-axis accelerometer and gyroscope data at 50Hz, buffering it to detect rapid Parkinson's resting tremors (4-6Hz) or user-initiated SOS hand gestures.

3.2 The Smart Belt (belt/main.py)

The chest belt monitors core metrics and provides emergency safety functions:

• **AD8232 ECG:** Captures single-lead electrocardiogram data at 250Hz to identify Arrhythmias and Premature Ventricular Contractions.

• **MPU6050 (Belt):** Continuously tracks torso orientation. Its primary function is Hard Fall Detection, calculating sudden acceleration spikes followed by horizontal resting states.

• **Physical SOS Button:** A physical interrupt button that instantly transmits a critical panic alert to the backend and cloud.

4. AI Models & Training

Each disease predictor acts as an independent module inside the backend/AI_models directory. The system utilizes Random Forest Classifiers and tailored architectures for rapid real-time inference.

4.1 Model Training Workflow

To retrain any model with new clinical data, execute its respective train module from the project root:

```
python -m AI_models.sepsis.train
```

This process automatically reads the local CSV dataset, performs a stratified cross-validation split, balances classes using sampling limits, and saves a .joblib or .keras bundle inside the model's saved_model directory alongside a detailed JSON metrics file.

4.2 Individual AI Modules

- **Arrhythmia:** Processes 250Hz ECG signals (AD8232). Identifies Premature Ventricular Contractions (PVC/VEB) vs Normal beats using engineered signal features.
- **Hypoxemia:** Monitors SpO2 and derives a smooth threshold score for dangerous dips below 92%.
- **Sepsis:** Relies on vital signs (HR, SpO2, Temp). Fuses signals via a Random Forest to detect early systemic shock signatures.
- **Parkinson's:** Tracks 50Hz 3-axis accelerometer data to classify distinct 5Hz resting hand tremors.
- **Facial Stress:** A multi-modal implementation combining a RandomForest trained on vital thresholds (HR/GSR) and a fine-tuned MobileNetV2 Facial Recognition neural network to classify distress via camera feed.

5. Advanced Backend Functions & Heuristics

Beyond Machine Learning, the system utilizes deterministic engines to fuse data, ensure safety, and cross-validate AI predictions. These are located in backend/AI_models/.

Multi-Sensor Correlation Engine (correlation_engine.py)

Cross-checks multiple models simultaneously to elevate priority. For instance, if the Sepsis AI triggers alongside a low SpO2 reading from the Hypoxemia model, the Correlation Engine merges them into a single, high-urgency 'Critical Multi-System Alert'.

Early Sepsis Detector (early_sepsis_detector.py)

A deterministic rules-engine utilizing the medical SIRS (Systemic Inflammatory Response Syndrome) criteria. It evaluates Tachycardia (HR > 90), Tachypnea, and Temperature thresholds to catch Sepsis even before the AI model gains confidence.

Silent Hypoxia Detector (silent_hypoxia.py)

Specifically monitors for 'Happy Hypoxia' where a dangerous drop in SpO2 (e.g. < 90%) occurs without the expected physiological compensatory spike in Heart Rate. This is a critical indicator of certain acute respiratory distress syndromes.

SOS Gesture Detector (sos_gesture.py)

Analyzes raw IMU arrays to identify the universal 'Signal for Help' (tucking the thumb and trapping it with fingers). Detects the specific rapid rolling sequence in the gyroscope data to silently trigger an alarm without physical buttons.

Dual-Node Sensor Fusion (sensor_fusion.py)

Synchronizes timestamps and integrates data between the Glove (extremities) and the Belt (core). Ensures that metrics like HR (from the glove) align with ECG rhythms (from the belt) to filter out artifacts and motion noise.

6. Backend Architecture & Inference API

The Python Flask backend (app.py) is the central processor. It receives JSON packets from the wearable hardware via local HTTP POST requests and streams it to the dashboard.

Processing Pipeline

- **Data Reception:** Hardware posts to /api/v1/telemetry.
- **Local Simulation:** If running in simulation mode, simulator.py intercepts requests and injects fast-forwarded clinical data.
- **Inference:** Both AI models and heuristic engines evaluate the incoming 1-second data window.
- **WebSockets:** The fully processed JSON payload is emitted via Flask-SocketIO to the local dashboard for live rendering.

7. Firebase & Cloud Integration

The backend maintains a persistent connection to Google Firebase Realtime Database. This synchronizes the edge computing node with the cloud, powering cross-platform mobile apps and remote caregiver dashboards.

Firebase Sync Workflow

In app.py, the firebase_admin SDK authenticates using the service account key. After the backend engine processes telemetry and updates local state:

```
ref = db.reference(f'users/{user_id}/telemetry')
ref.update(payload)
```

Cloud Data Structure

The JSON structure sent to Firebase is heavily optimized for fast mobile reads:

- vitals: Realtime snapshot of HR, SpO2, and Temp.
- alerts: High-priority trigger strings aggregated by the Correlation Engine.
- diseases: True/False binary flags for each specific AI model's output (sepsis_detected, hypoxemia_detected, etc).
- fall_detected: Boolean flag injected by the Belt's hardware interrupt.
- timestamp: Isoformat UTC stamp to ensure chronological sync and historical graphing.

This ensures healthcare providers can monitor patients securely over the cloud with near-zero latency.

End of Comprehensive User Guide. Ensure all Python dependencies listed in the core requirements file are installed before executing backend operations.